

Universidad Rafael Landívar

Ciencia de Datos · Segundo semestre 2026

Ing. Max Cerna

Proyecto 1 (Fase 1)

Red Metropolitana

Integrantes:

Yasmely Dubraska Fariñas Aldana 1329223

Jose Rodrigo Herrera Muralles 1042619

Walter José Herrera Salvatierra 1109722

23 de septiembre de 2026

Resumen

En esta fase armamos el pipeline completo por capas (Bronze → Staging → Silver → Gold) para juntar los datos de los cuatro operadores: Transmetro, Transurbano, MetroRiel y Aerómetro. Usamos las herramientas recomendadas del curso: Kafka para streaming (simulado en desarrollo), carpetas con Parquet para el lake, DuckDB como warehouse, dbt para las transformaciones y Prefect para orquestar.

Los datos son los 9 archivos que genera generar_red_metropolitana.py con la configuración por defecto (45 días desde el 1 de junio de 2026, unos 137 MB).

Qué medimos	Resultado
Filas ingeridas	1,730,184 (todas cuadran archivo = Bronze)
Padrón de Transmetro	15,611 activas antes de los borrados → 14,834 activas y 2,598 de baja después
Usuarios únicos	TM 43,255 · TU 36,567 · MR 22,885 · AM 14,496
Registros en cuarentena	12,336 (9,707 de operación + 2,629 del CDC)
Filas en Gold	1,647,569 abordajes y 295,511 trayectos de MetroRiel
Pruebas de dbt	108 de 108 pasan
Idempotencia	52 tablas, 0 diferencias entre la corrida 1 y la 2

Dónde está cada script

Punto	Script
1.1	ingesta/productor.py y consumidor.py (streaming), ingest/batch.py (catálogos, MetroRiel y Transurbano), ingest/cdc.py (CDC)
1.2	dbt/models/cdc/cdc_operaciones.sql, padron_vigente.sql, padron_transmetro_vigente.sql y cat_usuarios_*.sql
1.3	dbt/models/staging/ y dbt/models/silver/ (silver_zona, silver_identidad_usuario, silver_eventos_evaluados, silver_cuarentena, silver_padron_scd2)
1.4	docs/1.4-ddl-gold.sql y dbt/models/gold/
1.5	flows/pipeline.py (Prefect)

1.1 Ingesta y capa Bronze

Vía de cada archivo

Archivo	Vía	Por qué
transmetro_validaciones.csv	Streaming (Kafka)	Cada validación es un evento que llega todo el tiempo
aerometro_boardings.csv	Streaming (Kafka)	Igual, cada abordaje es un evento suelto
Los 4 catálogos	Batch	Casi nunca cambian
metroriel_viajes.jsonl	Batch	MetroRiel entrega el viaje ya cerrado
cdc_padron_usuarios.csv	CDC	Es lo único que se actualiza y se borra
transurbano_transacciones.csv	Batch (decisión nuestra)	Ver abajo

El streaming lo hacemos con un productor que publica el CSV línea por línea y un consumidor que lo escribe a Bronze. En desarrollo lo corrimos en modo simulado (el mismo productor y consumidor, pero conectados en memoria), porque no teníamos Docker. El código para Kafka real está listo (docker-compose.yml).

¿Por qué Transurbano va por batch?

- **Cada fila es una transacción de cobro, no un abordaje.** Trae un código de estado y 41,390 transacciones (5%) son pagos rechazados (saldo insuficiente o tarjeta inválida). Parece más una exportación del sistema de cobro que un flujo de torniquete.
- **No trae un ID de transacción.** Ojo, no hablamos del ID del usuario (ese sí viene), sino de un identificador de cada transacción. Sin eso no hay una llave para publicar cada mensaje en Kafka ni para detectar si un mensaje llegó dos veces. En batch identificamos el archivo completo por su hash.
- **El formato es de cierre de lote:** fecha y hora en columnas separadas y el monto en centavos.

Consecuencia de esta decisión: Transurbano llega con retraso (hasta un día) comparado con Transmetro y Aerómetro, que entran por streaming. Eso quiere decir que su demanda no se puede ver en vivo en el tablero, y que cualquier análisis de transbordo cercano en tiempo real (por ejemplo, alguien que baja del Transmetro y sube a un bus de Transurbano minutos después) queda incompleto hasta que entra la carga batch. Si en el futuro se necesitara en tiempo real, solo hay que cambiar su vía en `ingesta/comun.py`, pero primero habría que pedirle al operador que mande un ID de transacción.

¿Bronze vive en el lake o en el warehouse? En el lake

Bronze son carpetas con archivos Parquet:

lake/bronze/fuente=<fuente>/fecha_ingesta=<fecha>/<hash>.parquet. Lo decidimos así porque:

- **El JSON anidado de MetroRiel se guarda tal cual.** Parquet guarda entry y exit como estructuras anidadas sin aplanarlas, y los viajes sin salida quedan con exit en null.
- **Todo se guarda como texto.** Si viene una fecha mal escrita, Bronze la guarda como llegó y es Silver el que decide si va a cuarentena.
- **No depende del motor.** DuckDB lee el lake directo. Si mañana cambiamos a otro warehouse, Bronze no se toca.
- **Ocupa menos.** Los 136.8 MB crudos quedaron en 31.3 MB en Parquet.

Cada fila lleva la hora de ingesta (_ingested_at), el archivo de origen, el hash del archivo y la vía. La partición es por fecha de ingesta.

Tabla de conteos por archivo

Archivo	Vía	Filas en archivo	Filas en Bronze	¿Cuadra?
tm_estaciones.csv	batch	104	104	Sí
tu_paradas.csv	batch	328	328	Sí
mr_estaciones.csv	batch	22	22	Sí
am_estaciones.csv	batch	14	14	Sí
metroriel_viajes.jsonl	batch	299,100	299,100	Sí
transurbano_transacciones.csv	batch	832,791	832,791	Sí
cdc_padron_usuarios.csv	cdc	31,050	31,050	Sí
transmetro_validaciones.csv	streaming	363,221	363,221	Sí
aerometro_boardings.csv	streaming	203,554	203,554	Sí
Total		1,730,184	1,730,184	Sí

1.2 Staging y CDC

Staging es un esquema del warehouse que se borra al inicio de cada corrida (DROP SCHEMA staging) y dbt lo vuelve a llenar desde Bronze. Bronze en cambio nunca se borra, solo crece.

Hallazgo: el padrón no es solo de Transmetro

En teoría el archivo son los cambios del padrón de Transmetro, pero la columna tarjeta trae cuatro formatos:

Formato	Operador	Operaciones
TC-00012345	Transmetro	22,326
0000012345	Transurbano	5,223
MR0012345	MetroRiel	1,295
SIN-TARJETA	Ninguno	2,206

Lo que hicimos: lo tratamos como un registro central de personas donde cada tarjeta guarda de qué operador es, y sacamos aparte el padrón de Transmetro (solo tarjetas TC-), que es lo que pide el enunciado. Las filas SIN-TARJETA van a cuarentena, porque no se sabe de quién son; si las aplicáramos, todas se volverían una sola "tarjeta" que mezclaría a miles de personas.

Cómo aplicamos el CDC

Las operaciones se aplican en orden de secuencia, tarjeta por tarjeta:

Operación	Caso	Qué hacemos
INSERT	Tarjeta nueva	Queda activa
INSERT	Tarjeta que estaba de baja	Se reactiva
UPDATE	Tarjeta activa	Cambia perfil o zona de residencia
UPDATE	Primera operación de la tarjeta	La tarjeta ya existía antes del log, se toma como alta
UPDATE	Tarjeta de baja	No se aplica, va a cuarentena (CDC UPDATE SOBRE BAJA)
DELETE	Cualquiera	Baja lógica: activa = false y fecha de baja. La tarjeta NO se borra, así no se pierde el historial de sus viajes

Los DELETE sin cuerpo no van a cuarentena

Los DELETE llegan solo con la llave, sin perfil ni zona. Revisamos que eso no los mande a cuarentena por venir incompletos:

- Los 4,050 DELETE del archivo llegan sin cuerpo.
- Se aplicaron 3,772. Para no dejar la fila vacía, el perfil y la zona se copian de la última versión conocida de esa tarjeta.
- Los 278 que quedaron en cuarentena son solo los que traen SIN-TARJETA, o sea que no tienen ninguna llave. Ninguno se rechazó por venir sin cuerpo.
- DELETE con tarjeta válida que no se aplicaron: 0.

Conteo antes y después de aplicar los borrados

Métrica	Valor
Operaciones en el log	31,050
En cuarentena (2,206 SIN-TARJETA + 423 UPDATE sobre baja)	2,629
Operaciones aplicadas	28,421
Altas (INSERT)	10,003
Cambios (UPDATE)	14,646

Bajas (DELETE)	3,772
Reactivaciones	281

Por operador de la tarjeta:

Operador	Activas ANTES	Activas DESPUÉS	Dadas de baja	Total en padrón
Transmetro	15,611	14,834	2,598	17,432
Transurbano	3,638	3,430	608	4,038
MetroRiel	899	846	146	992
Total	20,148	19,110	3,352	22,462

Respuesta: Transmetro: antes de los borrados había 15,611 tarjetas activas. Después quedaron 14,834 activas y 2,598 dadas de baja.

El padrón tiene más tarjetas después de los borrados, no menos. Eso pasa porque hay tarjetas que solo aparecen en un DELETE (ya existían antes de que empezara el log) y quedan registradas de baja. Es la prueba de que el DELETE no borra.

Usuarios únicos por operador

Transurbano, MetroRiel y Aerómetro no tienen padrón. Para cada uno armamos un catálogo mínimo con las llaves distintas que aparecen en su archivo de operación. Solo tienen una columna (la llave), sin nombre ni fecha de alta ni nada más, porque esos operadores nunca mandaron esa información y no la vamos a inventar.

Operador	Archivo	Usuarios únicos
Transmetro	transmetro_validaciones.csv	43,255
Transurbano	transurbano_transacciones.csv	36,567
MetroRiel	metroriel_viajes.jsonl	22,885
Aerómetro	aerometro_boardings.csv	14,496

Transmetro aparece solo para comparar; él sí tiene padrón.

1.3 Capa Silver

Unificación de formatos

	Transmetro	Transurbano	MetroRiel	Aerómetro	En Silver
Fecha	YYYY-MM-DD HH:MM:SS	DD/MM/YYYY + hora aparte	ISO 8601	ISO en UTC	Timestamp en hora de Guatemala. Aerómetro se pasa de UTC a UTC-6
Dinero	Quetzales	Centavos	Quetzales	Quetzales	Todo en quetzales (Transurbano / 100)
Una fila es	Abordaje	Transacción	Viaje completo	Abordaje	TU: solo estados 1, 2 y 3 son abordajes. MR: la entrada es el abordaje

Dimensión zona

Cada fuente escribe la zona distinto (Zona 10, Z10, District 10, MIXCO, Mixco). Hicimos una función que lleva cualquier forma a una sola clave (Z10, MIXCO), sin tildes ni espacios, y la comparamos contra una lista oficial de 26 territorios: las 22 zonas de la capital (no existen la 20, 22 ni 23) y 4 municipios (Mixco, Villa Nueva, San Miguel Petapa y Santa Catarina Pinula). Aparecen 60 valores distintos en las fuentes y los 60 quedaron conformados. El detalle está en entregables/1.3-conformacion-zona.csv.

Identidad del usuario

No hay una tabla que relacione las cuatro tarjetas, así que cualquier estrategia es una aproximación. La nuestra:

- **TM, TU y MR se unen por el número.** TC-00012345, 0000012345 y MR0012345 se reducen a 12345 y los tomamos como la misma persona. Antes revisamos que el formato sea del operador correcto.
- **Aerómetro no se une.** Su llave es un hash de 12 caracteres que no trae el número de la persona, así que no hay con qué compararlo. Cada hash queda como una persona aparte.
- **No inventamos vínculos.** Lo que no se puede unir se queda solo.
- **Seudonimización antes de Gold.** usuario_sk = md5(sal secreta + persona). La sal está en el archivo .env, que no se sube al repositorio. Gold nunca ve la tarjeta.

Sistema	Llaves distintas	Unidas por número	Sin unir	Usan también otro sistema
Transmetro	43,257	43,257	0	32,920
Transurbano	36,567	36,567	0	30,308
MetroRiel	22,885	22,885	0	20,422
Aerómetro	14,496	—	14,496	0

Resultado: 70,380 personas; 36,829 (52%) usan más de un sistema (26,833 usan dos y 9,996 usan tres). En esta tabla Transmetro tiene 43,257 llaves y no 43,255 como en usuarios únicos, porque aquí también cuentan 2 tarjetas TC- del padrón que nunca viajaron.

Supuesto: TM, TU y MR usan la misma numeración de personas, o sea que el mismo número en dos sistemas es la misma persona. Lo apoya el padrón del CDC, que usa la llave de cualquiera de los tres para la misma persona.

Límites:

- Los usuarios de Aerómetro cuentan como personas separadas. Eso infla el total de personas y esconde parte del transbordo.
- Si dos operadores usaran el mismo número para personas distintas, las uniríamos mal. Sin un padrón común no se puede detectar.
- Si una persona tiene dos tarjetas del mismo sistema, cuenta como dos personas.

Para medir cuánto afecta no unir Aerómetro, probamos unirlo con la regla del hash que aparece en el script generador (una Agencia real no la tendría, por eso no es el resultado oficial). Con eso las personas bajan de 70,380 a 56,848 y los multisistema suben de 36,829 a 41,486. O sea, no unir Aerómetro sobreestima las personas en 24%. Es un buen argumento para pedirle a Aerómetro su regla.

Reglas de calidad y cuarentena

Los registros malos no se botan: van a la tabla silver_cuarentena con su motivo, una explicación, el registro original completo y el archivo de donde vino. Cada registro tiene un solo motivo: el de la primera regla que no cumple, en este orden:

#	Regla	Cuándo aplica
1	DUPLICADO_REINGESTA	El mismo evento llegó en otro archivo
2	DUPLICADO_TORNIQUETE	El mismo evento sale dos veces en el mismo archivo (el torniquete leyó dos veces)
3	USUARIO_NULO	No trae llave de usuario
4	PARADA_NULA	Código de parada o estación vacío
5	ESTACION_DESCONOCIDA	La estación no está en el catálogo
6	FECHA_INVALIDA	La fecha no se puede leer
7	FECHA_FUTURA	La fecha es después de cuando se ingirió el registro
8	VIAJE_SIN_SALIDA	Viaje de MetroRiel sin salida (exit en null)
9	SALIDA_INCONSISTENTE	Salida antes de la entrada o en la misma estación
10	ZONA_INVALIDA	La estación no tiene una zona reconocida
11	MONTO_INVALIDO	Monto vacío o negativo
12	ESTADO_DESCONOCIDO	Estado de Transurbano que no es 1, 2, 3, 7 ni 9
CDC	CDC_SIN_TARJETA / CDC_UPDATE_SOBRE_BAJA	Ver sección 1.2

Las transacciones de Transurbano con estado 7 o 9 no son cuarentena: son pagos rechazados que están bien registrados. No cuentan como abordaje, pero las guardamos en silver_transacciones_rechazadas_tu (41,390).

Conteo de registros por regla

Fuente	Regla	Registros	% de la fuente
Transmetro	DUPLICADO_TORNIQUETE	1,115	0.31%
Transurbano	PARADA_NULA	4,189	0.50%
Transurbano	FECHA_FUTURA	814	0.10%
MetroRiel	VIAJE_SIN_SALIDA	3,589	1.20%
CDC	CDC_SIN_TARJETA	2,206	7.11%
CDC	CDC_UPDATE_SOBRE_BAJA	423	1.36%
Total		12,336	

Salen justo los cuatro problemas que menciona el enunciado: duplicados de torniquete, paradas nulas, fechas del futuro y viajes sin salida. Aerómetro no tuvo ningún registro malo. Las otras reglas están programadas pero con estos datos no se activaron. En total, los 9,707 registros de operación en cuarentena son el 0.57% de 1,698,666. Duplicados detectados: 1,115 de torniquete y 0 de reingesta.

La tabla de cuarentena completa (12,336 registros con motivo) está en entregables/1.3-cuarentena.csv. Aquí van unos ejemplos:

Fuente	Evento	Motivo	Detalle
CDC	1000	CDC_SIN_TARJETA	La operación no trae una tarjeta identificable (SIN-TARJETA)
CDC	10088	CDC_UPDATE_SOBRE_BAJA	UPDATE sobre una tarjeta que ya estaba dada de baja: no se aplica
MetroRiel	100046	VIAJE_SIN_SALIDA	Viaje de MetroRiel sin validación de salida
Transmetro	10008	DUPLICADO_TORNIQUETE	Lectura repetida: mismo evento dos veces en el mismo archivo (se conserva la pri
Transurbano	0070b737bb0e3d	FECHA_FUTURA	Fecha 2027-08-17 06:54:16 posterior a la ingesta 2026-09-23 14:31:32.223327
Transurbano	00005289057eb4	PARADA_NULA	Código de estación/parada vacío

No se pierde ningún registro

Sistema	Filas en Bronze	Abordajes limpios	Pagos rechazados	Cuarentena
Transmetro	363,221	362,106	—	1,115
Transurbano	832,791	786,398	41,390	5,003
MetroRiel	299,100	295,511	—	3,589
Aerómetro	203,554	203,554	—	0
Total	1,698,666	1,647,569	41,390	9,707

Esto lo revisa una prueba de dbt (assert_ningun_registro_se_pierde) en cada corrida.

Padrón con SCD Tipo 2

silver_padron_scd2 tiene una fila por cada versión de cada tarjeta, con valido_desde, valido_hasta (9999-12-31 si está vigente) y es_vigente. Cada operación aplicada abre una versión nueva y cierra la anterior; el DELETE abre una versión con activa = false. Quedaron 28,421 versiones para 22,462 tarjetas. Con esto se puede saber, por ejemplo, qué perfil tenía una tarjeta el 20 de junio.

1.4 Diseño dimensional y capa Gold

Grano de la tabla principal

fct_abordaje: una fila por cada abordaje de un usuario en una estación, de cualquiera de los cuatro sistemas.

Elegimos el abordaje porque es lo único que registran los cuatro. Si usáramos "viaje puerta a puerta" tendríamos que inventar los viajes de tres sistemas encadenando validaciones, y eso mete supuestos en la tabla principal.

MetroRiel de cada viaje de MetroRiel sacamos su abordaje (estación y hora de entrada) y lo metemos en fct_abordaje, igual que los otros tres. Así la demanda por modo, zona y hora está completa con los cuatro modos (295,511 abordajes de MetroRiel). El viaje completo (salida, duración, estaciones recorridas) no se pierde: va en una segunda tabla de hechos, fct_trayecto_metroriel, con su propio grano.

Sistema	Filas en fct_abordaje
Transmetro	362,106
Transurbano	786,398
MetroRiel	295,511
Aerómetro	203,554
Total	1,647,569

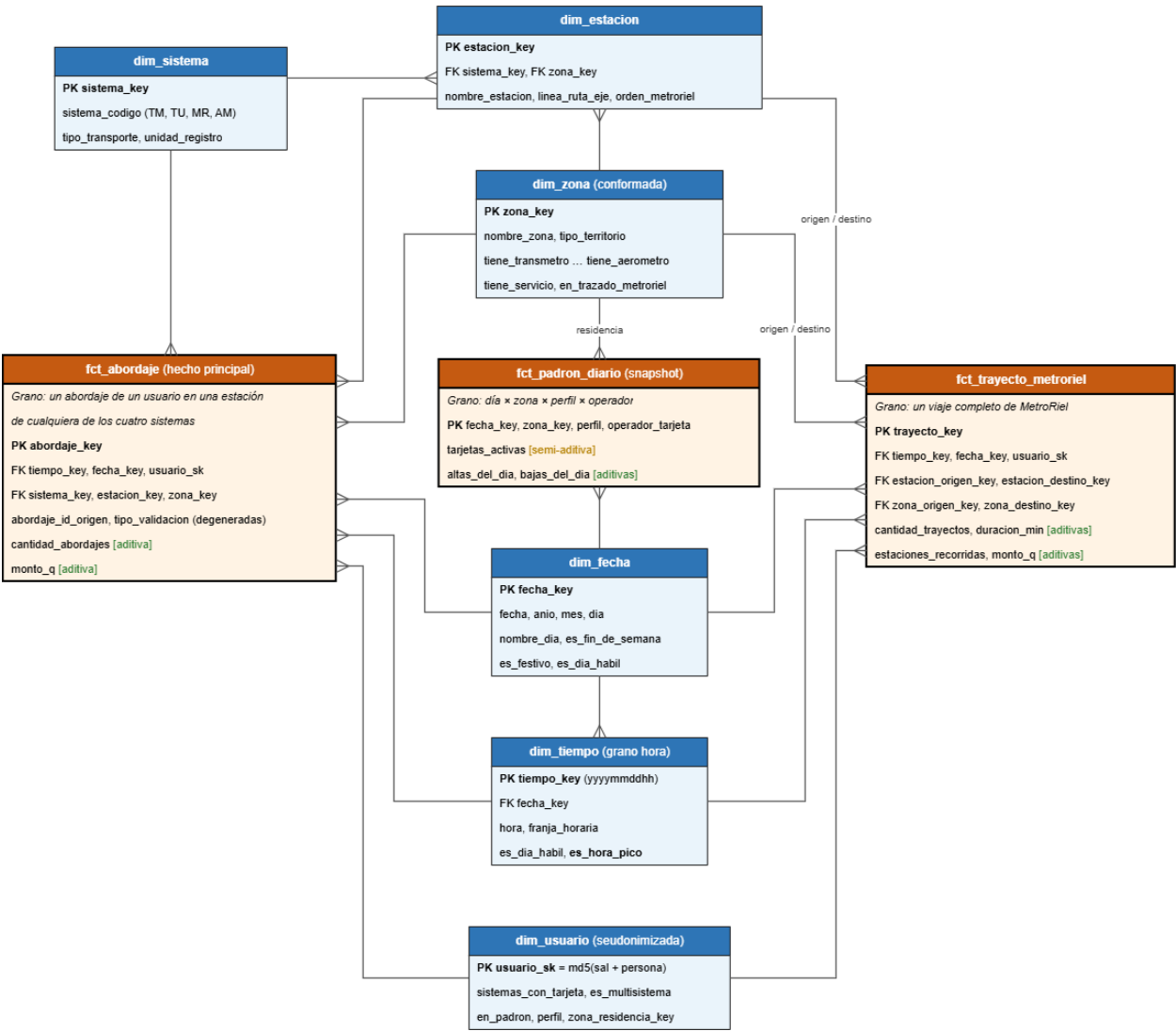
Matriz del bus

Proceso (tabla de hechos)	Fecha	Tiempo	Sistema	Estación	Zona	Usuario
Abordajes (fct_abordaje)	✓	✓	✓	✓	✓	✓
Trayectos MetroRiel (fct_trayecto_metroriel)	✓	✓	✓ (MR)	✓ origen y destino	✓ origen y destino	✓
Padrón de tarjetas (fct_padron_diario)	✓		✓ operador		✓ residencia	

Las dimensiones son las mismas tablas para los tres procesos (conformadas): dim_fecha, dim_tiempo, dim_sistema, dim_estacion, dim_zona y dim_usuario.

Diagrama del modelo

Capa Gold · Modelo dimensional (esquema estrella)
Red Metropolitana de Transporte



Dimensión tiempo

- dim_tiempo tiene grano de hora (8,760 filas para 2026) y se relaciona con dim_fecha (grano día).
- Hora del día (0 a 23) y franja (madrugada, mañana, tarde, noche).
- Día hábil: lunes a viernes que no sea asueto. En el periodo de los datos cae el 30 de junio (Día del Ejército).
- Hora pico: día hábil y hora entre 6:00 y 8:59 o entre 17:00 y 19:59.

Clasificación de medidas

Medida	Tabla	Tipo	Por qué
cantidad_abordajes	fct_abordaje	Aditiva	Se suma por cualquier dimensión
monto_q	fct_abordaje, fct_trayecto_metroriel	Aditiva	El ingreso se suma por cualquier dimensión
cantidad_trayectos,	fct_trayecto_metroriel	Aditiva	Se suman (el promedio no)

duracion_min, estaciones_recorridas			
altas_del_día, bajas_del_día	fct_padron_diario	Aditiva	Son movimientos del día, se suman entre días
tarjetas_activas	fct_padron_diario	Semi-aditiva	Es un saldo: se suma entre zonas u operadores, pero no entre días (19,000 el lunes + 19,000 el martes no son 38,000)
Usuarios distintos	Calculada	No aditiva	Un usuario que viaja en dos zonas se contaría dos veces
Duración promedio, % en hora pico	Calculada	No aditiva	Son cocientes, se recalculan en cada nivel

DDL y filas en Gold

El DDL completo con llaves primarias y foráneas está en docs/1.4-ddl-gold.sql. Lo probamos corriéndolo en una base vacía. Ningún modelo de Gold lee de Bronze: el flujo lo revisa en cada corrida y se detiene si pasa.

Tabla	Filas
dim_estacion	468
dim_fecha	365
dim_sistema	4
dim_tiempo	8,760
dim_usuario	70,380
dim_zona	26
fct_abordaje	1,647,569
fct_padron_diario	8,612
fct_trayecto_metroriel	295,511

dim_zona ya trae la cobertura: 15 territorios tienen al menos una estación y 11 no tienen servicio de ningún modo (zonas 2, 3, 5, 14, 15, 16, 19, 21, 24 y 25, y Santa Catarina Pinula).

1.5 Orquestación e idempotencia

El flujo está en flows/pipeline.py con Prefect: ingesta batch → CDC → streaming → vaciar staging → dbt build (modelos y 108 pruebas) → revisar linaje → contar → exportar entregables.

Por qué no duplica

- **Bronze:** cada archivo se identifica por su hash. Si ya se cargó, se salta.
- **Kafka:** cada mensaje lleva la llave <hash>:<línea>, así el consumidor ignora los repetidos.
- **Staging:** se borra al inicio de cada corrida.
- **Silver y Gold:** son tablas que dbt reconstruye desde Bronze; mismo Bronze, mismo resultado. FECHA_FUTURA compara contra la hora de ingesta y no contra la hora actual, para que no cambie entre corridas.

Evidencia: dos corridas con conteos idénticos

Corrimos el flujo desde cero (2026-09-23 14:32:33), contamos, lo volvimos a correr (2026-09-23 14:33:25) y contamos otra vez. Resultado de herramientas/comparar.py:

52 tablas comparadas, 0 diferencias -> IDENTICOS

En la corrida 2 los 9 archivos salen como "ya ingerido, se omite". Algunas tablas:

Capa	Tabla	Corrida 1	Corrida 2	¿Igual?
bronze	transmetro_validaciones	363,221	363,221	Sí
bronze	transurbano_transacciones	832,791	832,791	Sí
bronze	metroriel_viajes	299,100	299,100	Sí
bronze	aerometro_boardings	203,554	203,554	Sí
bronze	cdc_padron_usuarios	31,050	31,050	Sí
control	ingest_log	9	9	Sí
silver	silver_eventos_evaluados	1,698,666	1,698,666	Sí
silver	silver_cuarentena	12,336	12,336	Sí
silver	silver_abordajes	1,647,569	1,647,569	Sí
silver	padron_vigente	22,462	22,462	Sí
silver	silver_padron_scd2	28,421	28,421	Sí
gold	fct_abordaje	1,647,569	1,647,569	Sí
gold	fct_trayecto_metroriel	295,511	295,511	Sí
gold	fct_padron_diario	8,612	8,612	Sí
gold	dim_usuario	70,380	70,380	Sí

Las 52 tablas están en evidencia/comparacion-corridas.csv.

Rendimiento medido

Etapa	Corrida 1 (s)	Corrida 2 (s)
Bronze batch	1.39	0.24
Bronze CDC	0.22	0.04
Bronze streaming (simulado)	12.53	0.10
Vaciar staging	0.03	0.06
dbt build (modelos + 108 pruebas)	47.99	35.41
Consulta del tablero (modo × zona × hora)	23.5 ms	31.7 ms

Capa	Tamaño en disco
Archivos crudos	136.8 MB
Bronze (Parquet)	31.3 MB

Warehouse (staging + silver + gold)	499 MB
-------------------------------------	--------

El archivo de DuckDB crece un poco en la segunda corrida, pero no es porque se dupliquen datos (los conteos son iguales): DuckDB no libera el espacio de las tablas que dbt reemplaza.